

Lab ใช้ PID ในควบคุม Gimbal

การควบคุม Gimbal (กิมบอล) ของหุ่นยนต์ DJI RoboMaster (เช่น รุ่น S1 หรือ EP) ด้วย Python SDK เป็นพื้นฐานสำคัญสำหรับการทำระบบเล็งเป้าอัตโนมัติหรือการควบคุมกล้องเพื่อตรวจจับวัตถุ

Gimbal ของ RoboMaster จะมีทิศทางการเคลื่อนที่หลักอยู่ 2 แกน:

- Pitch (แกน X): การก้มหรือเงยกล้องและตัวยิง (องศาบวกคือเงยขึ้น องศาลบคือก้มลง)
- Yaw (แกน Z): การหันซ้ายหรือขวา (องศาบวกคือหันขวา องศาลบคือหันซ้าย)

จุดอ้างอิง (พิกัด 0,0) ของ Gimbal โดยปกติจะอ้างอิงจากตำแหน่งเริ่มต้น (Power-on) หรือตำแหน่งที่หันตรงกับด้านหน้าของแชสซี (Chassis)

ฟังก์ชันที่สำคัญ (Key APIs)

ในไลบรารี robomaster ชุดคำสั่งสำหรับควบคุม Gimbal จะอยู่ในโมดูล robot.gimbal:

- recenter() สั่งให้ Gimbal หันกลับมายังจุดกึ่งกลาง (จุดศูนย์) ของหุ่นยนต์
- moveto(pitch_angle, yaw_angle, pitch_speed, yaw_speed)

สั่งเคลื่อนที่ไปยัง "พิกัดสัมบูรณ์" (Absolute Position) เทียบกับจุดกึ่งกลาง

- move(pitch_angle, yaw_angle, pitch_speed, yaw_speed)

สั่งเคลื่อนที่แบบ "พิกัดสัมพัทธ์" (Relative Position) โดยขยับเพิ่มหรือลดองศาจาก *ตำแหน่งปัจจุบัน*

คำแนะนำ: การต่อท้ายด้วยฟังก์ชัน .wait_for_completed() จะทำให้โปรแกรมรอนจนกว่า Gimbal จะหมุนเสร็จ จึงค่อยทำงานในบรรทัดถัดไป

ใน Lab นี้เราจะใช้ การยิงอินฟราเรด (IR) เป็นหลักเพื่อความปลอดภัยและไม่เลอะเทอะ แต่สามารถเปลี่ยนเป็นกระสุนเจล (Water Bead) ได้

ฟังก์ชันที่สำคัญสำหรับปืน (Blaster)

- fire(fire_type, times) สั่งยิงปืน โดยกำหนดประเภทของกระสุนและจำนวนนัด
 - fire_type: ใช้ blaster.IR_FIRE (ยิงแสงอินฟราเรด/เสียง) หรือ blaster.WATER_FIRE (ยิงกระสุนเจล)
 - times: จำนวนนัดที่ต้องการยิงใน 1 ครั้ง

คำสั่งในโมดูล vision ของ RoboMaster SDK ทำหน้าที่เรียกใช้ AI ประมวลผลภาพที่ฝังอยู่ในตัวหุ่นยนต์

1. การเปิดใช้งานและรับข้อมูล (Subscribe) `ep_vision.sub_detect_info(name="marker", callback=on_detect_marker)`

- name: ระบุประเภทของวัตถุที่ต้องการให้ AI ตรวจสอบ ในที่นี้คือ "marker" (ป้าย Vision Marker) คำอื่นๆ ที่ระบบรองรับ ได้แก่ "person" (คน), "robot" (หุ่นยนต์ตัวอื่น) และ "gesture" (ท่าทางมือ)
- callback: ระบุชื่อฟังก์ชันที่จะถูกเรียกทำงานอัตโนมัติ (Event-driven) ทันทีที่กล้องมองเห็นวัตถุตามที่กำหนด โดยไม่ขัดจังหวะการทำงานของโค้ดหลัก

2. โครงสร้างข้อมูลที่กลับมา (Callback Data) เมื่อฟังก์ชัน `on_detect_marker(marker_info)` ทำงาน ตัวแปร `marker_info` จะเป็น List ที่บรรจุข้อมูลของวัตถุทั้งหมดที่ตรวจพบในเฟรมภาพนั้น โดยจะถูกแยกส่วนประกอบ (Unpack) ด้วยคำสั่ง: `x, y, w, h, info = marker`

- x, y: พิกัดจุดกึ่งกลางของป้ายเป้าหมาย (เป็นสัดส่วน Normalized จาก 0.0 ถึง 1.0)
 - แกน X: ซ้ายขอบจอคือ 0.0 ขวาขอบจอคือ 1.0 (ตรงกลางจอคือ 0.5)
 - แกน Y: บนขอบจอคือ 0.0 ล่างขอบจอคือ 1.0 (ตรงกลางจอคือ 0.5)
- w, h: ความกว้าง (Width) และความสูง (Height) ของเป้าหมายเมื่อเทียบกับขนาดจอทั้งหมด (ค่า 0.0 ถึง 1.0 เช่นกัน)
- info: ข้อมูลสัญลักษณ์ที่อ่านได้จากป้าย คืนค่ากลับมาเป็น String เช่น "1", "2", "A", หรือ "heart" ทำให้เราเขียนเงื่อนไข `if info == "1":` เพื่อเลือกถือเป้าหมายเฉพาะเบอร์ที่ต้องการได้

3. การยกเลิกการตรวจจับ (Unsubscribe) `ep_vision.unsub_detect_info(name="marker")`

- ใช้สำหรับปิดระบบ Vision เมื่อจบโปรแกรม หรือเมื่อต้องการเปลี่ยนไปตรวจจับวัตถุประเภทอื่น การสั่งยกเลิกเป็นการคืนทรัพยากร (RAM และ CPU) ให้ตัวหุ่นยนต์ เพื่อป้องกันอาการค้างหรือกระตุก

การทดลอง

ให้นักศึกษาเขียนโปรแกรม ควบคุม gimbal ให้ป็น ยิงไปยังเป้า 3 เป้าให้เร็วที่สุด และแม่นยำมากที่สุด (โดนกลางเป้า ใช้เป้ากระดาษรูปหัวใจ ระยะห่างระหว่างเป้า 0.6m ระยะห่างระหว่างหุ่นยนต์กับเป้ากกลาง 1m) โดยยิงจากเป้าซ้ายสุด วนไปขวาสุด และวนกลับมาเป้าซ้ายสุดอีกครั้ง โปรแกรมตัวอย่าง อยู่ใน LMS นักศึกษา สามารถดาวน์โหลดมา แล้วทำการ Tunning ตัวควบคุม แบบ PID ตามที่ได้เรียนมา หลังจาก ทดลองเสร็จแล้ว ให้นักศึกษา อัดวิดีโอ และ plot time response มุมของ gimbal เขียนรายงานส่ง